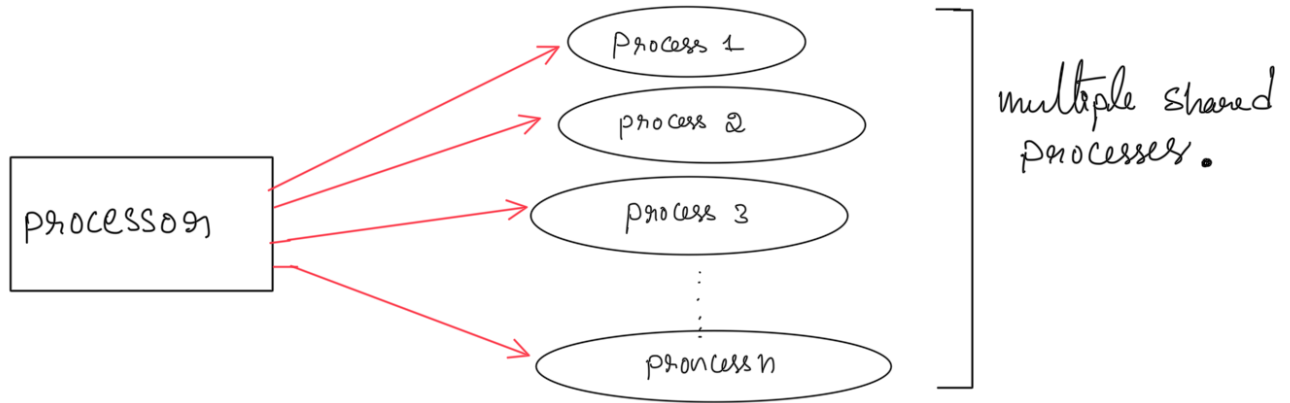


Process and Thread Scheduling in Operating System.

DAY 56
4/09/2025

Process is a Combined Shared Resource



We can observe in the above Block diagram there is a single processor and multiple shared process's, How each process get chance for execution, I mean how a single process will be handling multiple process's

If the system/processor runs the processes one by one the Multitasking becomes a Big challenge

A processor can't keep the other processes in the wait state until the particular process is executed and terminated in that case Multitasking becomes an issue for the processor

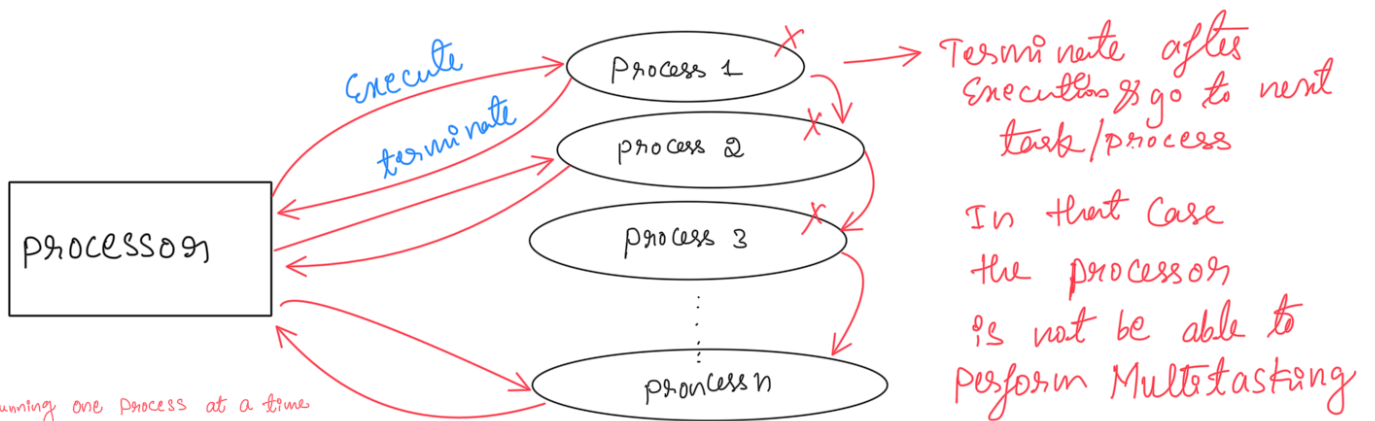
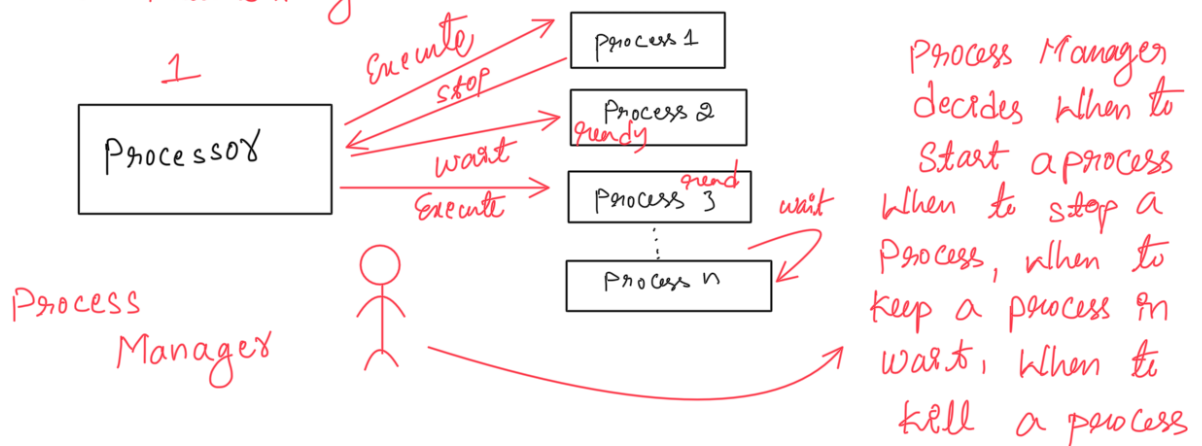


fig:-> Running one process at a time

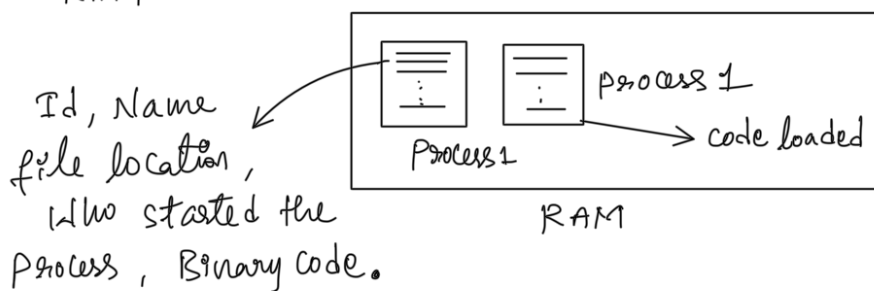
So we needed the Process Manager to run the multiple processes to achieve multitasking



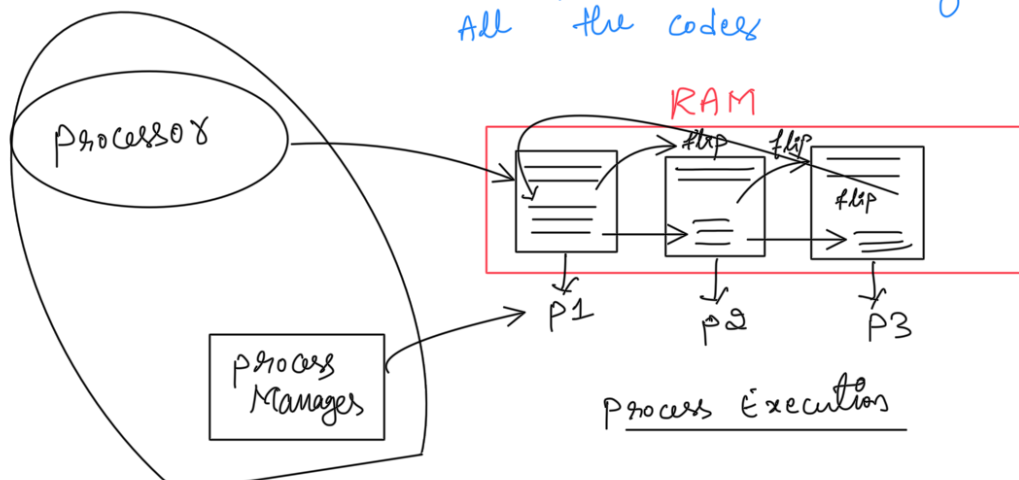
We have many processors and we have one processor. So the process manager will manage & run the process

Process have input/output will go into wait state in between

As we know that the process is logically mapped in the RAM



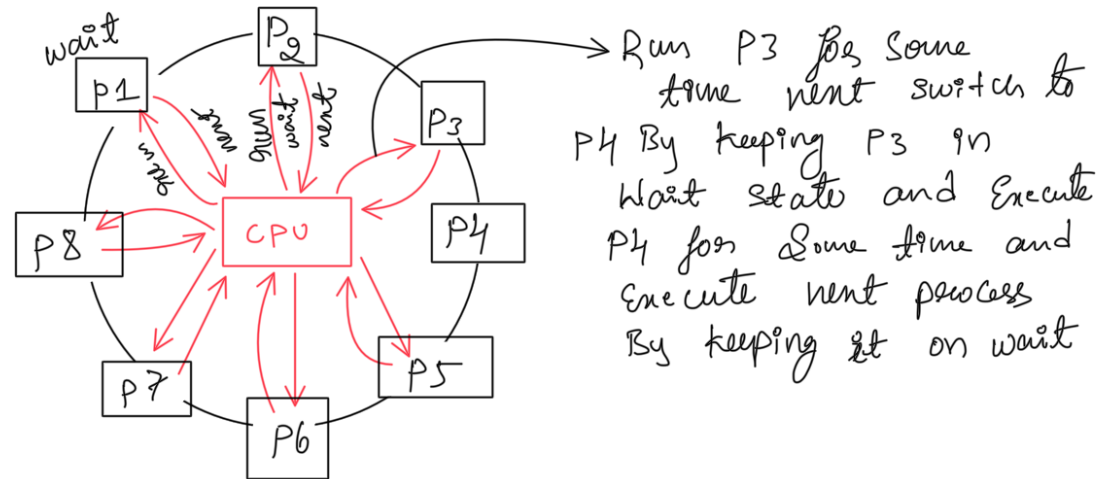
The processor is running and flipping with all the codes



To manage all of this flips we have Process Manager

Process Manager use different Algorithms In different Operating Systems.

Simple Algorithm is Round Robin



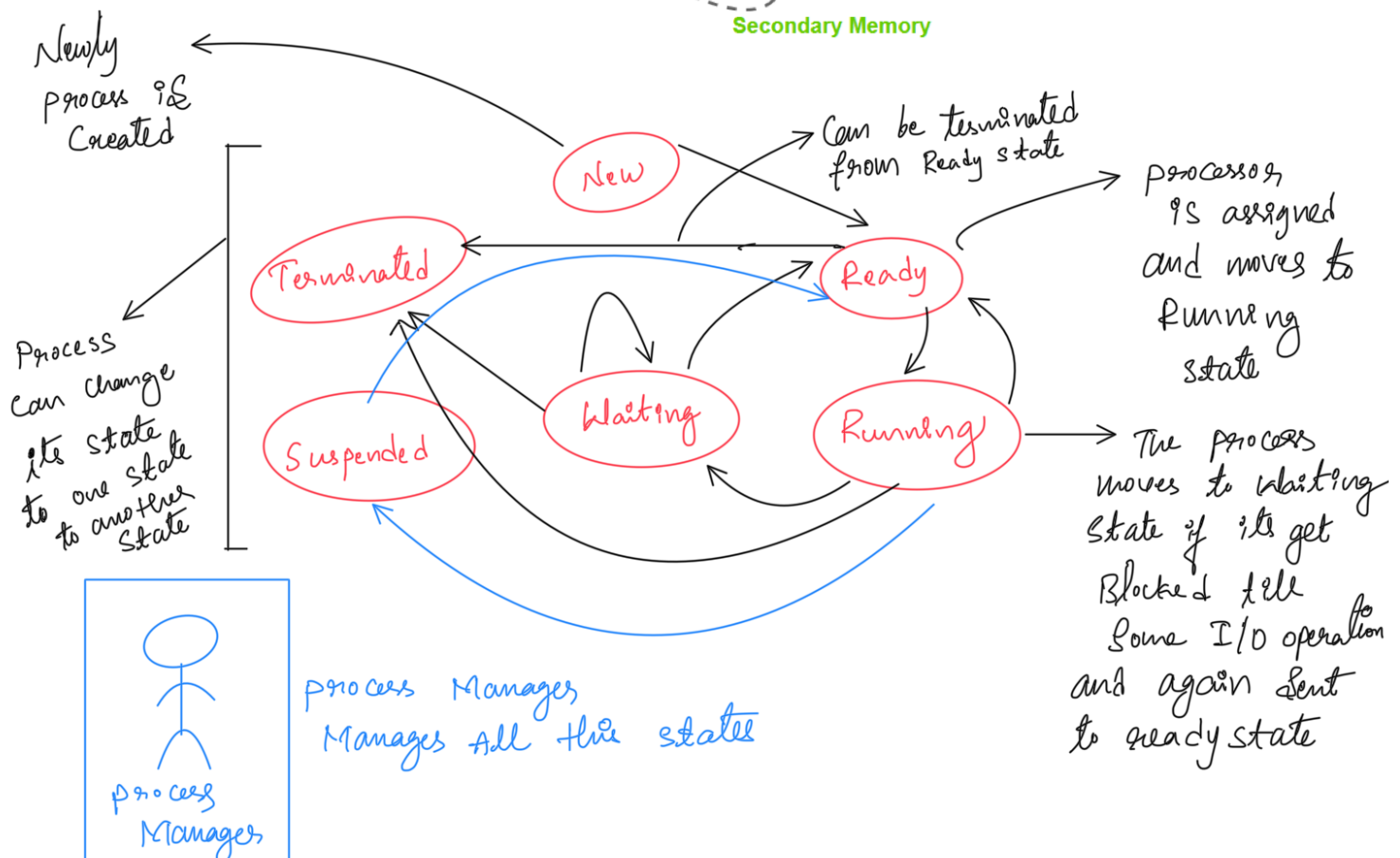
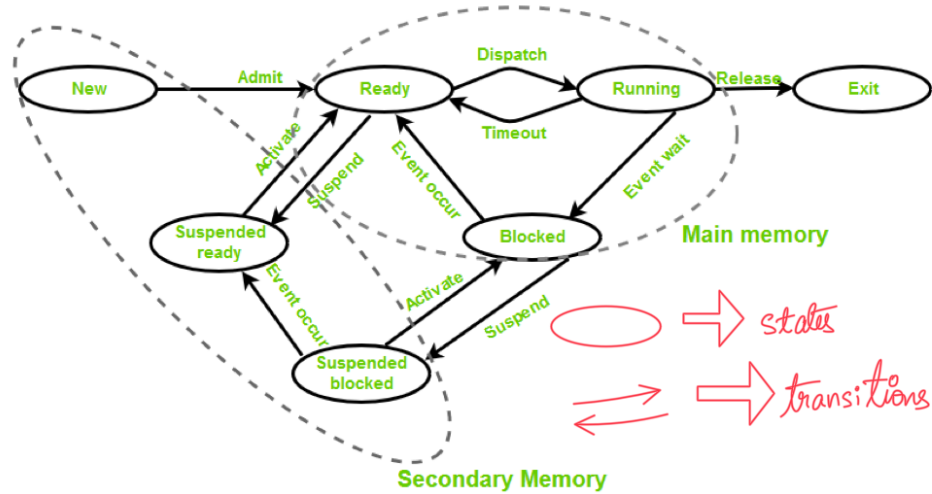
This way of solving problem of process execution is called as Round Robin

Process states

A process goes through the following states during its life cycle

- New → Process is being created (process added to Ready Queue)
- Ready → Process is waiting to be assigned to a processor
- Running → Instructions are being executed
- Waiting (Blocked) → Waiting for some event (I/O completion)
- Terminated → Process has finished execution
- Suspended → Process is not ready / running and is swapped out.

Process state → defines in which condition a particular process is there



◆ Suspended State

- A process is temporarily stopped by the operating system.
- It's not running and not ready (even if CPU is free, scheduler won't pick it).
- Why?
 - Process is waiting for some external event (e.g., user input, I/O completion).
 - Process swapped out to disk to free RAM (suspended by OS).
 - User/OS explicitly suspended (e.g., `kill -STOP` in Linux).

• Key Point:

◆ Terminated State

- A process has finished execution (either normally or killed).
- Its resources are released:
 - Memory freed
 - Files closed
 - PCB removed from the process table
- Why?
 - Program completed successfully.
 - User/OS killed it (`kill -9`, End Task).

Key Point:

- The process is still alive, has a PCB entry, and can be resumed (moved back to ready state).

👉 Analogy: Like putting a movie on *pause*. It still exists, you can resume later.

- Error/exception occurred.

Key Point:

- The process is **dead**. It cannot be resumed.

👉 Analogy: Like finishing a movie and ejecting the DVD — it's gone, can't resume.

Comparison Between
process Suspended &
process terminated

Quick Comparison

Aspect	Suspended	Terminated
Alive?	✅ Yes (can be resumed)	❌ No
PCB exists?	✅ Yes	❌ No (removed)
In RAM?	Maybe (or swapped to disk)	❌ No
Resume possible?	✅ Yes	❌ No
Analogy	Paused movie	Movie finished/ejected

4 Waiting → Ready

- When I/O completes, the process is moved back to the ready queue.

👉 Handled by: Interrupts (I/O completion interrupt notifies OS).

5 Running → Ready (Preemption)

- In multitasking OS, the scheduler may forcibly stop a process if:
 - Its **time slice** expires.
 - A higher-priority process arrives.
- The process goes back to the **ready queue**.

👉 Handled by: CPU scheduler + timer interrupt.

6 Running → Suspended

- A running process may be **suspended** due to:
 - User/OS request (`kill -STOP` in Linux).
 - Memory pressure (swapped out to disk).
- It's paused and won't run until resumed.

👉 Handled by: Process Manager + Memory Manager.

How Process Manager Handles Process States

1 New → Ready

- When you start a program, OS creates a **Process Control Block (PCB)**.
- The process is admitted into the **ready queue**.

👉 Handled by: Process creation system calls (`fork`, `exec`, etc. in UNIX).

2 Ready → Running

- The **CPU scheduler** picks a process from the ready queue.
- Context switch: PCB state is loaded into CPU registers, program counter set.

👉 Handled by: Short-term scheduler (dispatcher).

3 Running → Waiting (Blocked)

- If a process requests I/O (e.g., file read, user input), it **can't continue** until I/O finishes.
- Process Manager marks it as **waiting** and moves it to the I/O queue.

👉 Handled by: I/O subsystem + Process Manager.

7 Suspended → Ready

- If resumed (`kill -CONT`), it goes back to the ready queue.

👉 Handled by: OS signal handling.

8 Running → Terminated

- Process finishes execution or is killed.
- PCB removed, memory/resources freed, entry removed from process table.

👉 Handled by: Process Manager + Resource Manager.

Summary Table

State Transition	Handled By
New → Ready	Process creation system calls
Ready → Running	CPU Scheduler (Dispatcher)
Running → Waiting	I/O subsystem + Process Manager
Waiting → Ready	I/O Interrupt Handler
Running → Ready	CPU Scheduler (Preemption)
Running → Suspended	Process Manager / Memory Manager
Suspended → Ready	OS Signals
Running → Terminated	Process Manager + Resource Manager

✓ In short:

The Process Manager + Scheduler are like the traffic police of the CPU — they decide who runs, who waits, who gets paused, and who's thrown out.



This thing is implemented using process table

PID	PName	PLocation	Status	
			Running	
			Suspended	
			Ready	
			wait	
			terminated	

For Each process
its status
will be stored
In process
table

CPU

The process which is
in ready state for
that process CPU will
be allocated

Tomorrow I will learn this Process Management
and Process Scheduling Algorithms In depth
See you tomorrow.